Research Reference: StarForth Physics Runtime

Volume III of the StarshipOS Technical Reference

Robert A. James

Contents

Chapter 1

Published Paper: Steady-State Convergence

1.1 Citation

The primary experimental results are published in:

?

The paper is available at `docs/working/papers/published/James_Steady-State_Convergence_Adapt`

1.2 Abstract

1.3 Key Findings

1.4 Executive Summary

1.5 Executive Summary

1.5.1 The Problem

Modern runtime systems face a persistent conflict between two design goals. Adaptive systems—contemporary web browsers, mobile application runtimes, JIT-compiled virtual machines—learn which code paths run most often and reorganize themselves to accelerate those paths. They improve over time, but their internal decisions vary from run to run; two executions of the same program on the same machine may produce different optimization states. This non-determinism is acceptable for web browsers but disqualifying for safety-critical software, financial systems requiring auditable behavior, or real-time systems with timing guarantees.

Deterministic systems—firmware for medical devices, aircraft flight controllers, certified real-time kernels—run the same way on every invocation. They can be formally verified, exhaustively tested, and certified by standards bodies. But they cannot adapt: the optimization configuration frozen at compile time may be far from optimal for the actual workload seen at runtime.

StarForth resolves this conflict. The system is simultaneously adaptive and completely predictable at the level of its algorithmic decisions.

1.5.2 What Was Built

StarForth is a FORTH-79 compliant virtual machine with a physics-driven adaptive runtime. Using execution frequency as a proxy for thermal energy (a deliberate conceptual metaphor, not

a physics claim), the system tracks how often each dictionary entry executes, applies exponential decay to reduce the weight of stale executions, and maintains a small cache of the most frequently executed words for accelerated lookup. Every decision in this pipeline is deterministic: given the same execution sequence, the system produces the same cache configuration on every run.

Seven feedback loops coordinate the adaptive behavior:

- Execution heat tracking and hot-words caching (Loop 1)
- Rolling window of execution history (Loop 2)
- Linear decay of quiescent entries (Loop 3)
- Word-to-word transition prediction (Loop 4)
- Variance-based window width inference via Levene's test (Loop 5)
- Decay slope inference via exponential regression (Loop 6)
- Adaptive heartbeat coordination (Loop 7)

All loops are independently togglable and formally characterized. Fixed-point arithmetic ($\mathbb{Q}_{48.16}$ throughout) eliminates IEEE-754 non-determinism across architectures.

1.5.3 Experimental Results

A three-configuration design of experiments ran 90 trials (30 runs per configuration):

- **C_NONE** (baseline): all adaptive loops disabled
- **C_CACHE**: hot-words cache enabled, inference disabled
- **C_FULL**: all seven loops active

Result 1: algorithmic determinism. Across all 30 runs of C_FULL, the cache hit rate was 17.39% with a coefficient of variation (CV) of exactly 0.00%. The F-test for variance homogeneity yields $F(29, 29) \to \infty$, $p < 10^{-30}$. Even under intentional thermal stress (sustained CPU load), the algorithmic decisions did not change; only wall-clock runtime increased.

Result 2: adaptive convergence. C_FULL improved from a mean of 10.20 ms in early runs (1–15) to 7.61 ms in late runs (16–30), a statistically significant 25.4% improvement ($t = 4.23$, $p < 0.001$, Cohen's $d \approx 5.08$). The baseline configuration (C_NONE) showed slight degradation over the same window; C_CACHE showed no meaningful improvement. Only the fully adaptive configuration converged.

Result 3: variance decomposition. Runtime exhibits 60–70% CV due to OS scheduler noise, thermal variation, and cache-line effects. Cache decisions exhibit 0.00% CV. The two components are statistically independent (Pearson $r = 0.03$, $p = 0.87$). The adaptive algorithm is perfectly stable; the environment adds noise on top of a deterministic foundation.

1.5.4 Implications

Verifiable adaptive systems. A system whose adaptive decisions are deterministic can, in principle, be formally verified. The optimization state reachable from a given workload is predictable, auditable, and reproducible—properties required for safety-critical certification.

Reproducible performance characterization. Benchmark results become portable: different researchers running the same workload reach the same adaptive steady state, enabling apples-to-apples comparison of optimization strategies.

Adaptation without non-determinism. The results demonstrate that statistical inference can guide optimization without introducing randomness. The system adapts through arithmetic and counting, not machine learning or stochastic search.

1.5.5 Scope and Limitations

This work does not claim that StarForth is the fastest virtual machine, that its approach is superior to JIT compilation for all use cases, or that the thermodynamic framework is a physical theory. The 0.00% algorithmic CV applies to cache decisions, not to total system runtime. The system is validated on CPU-bound, deterministic FORTH programs; I/O-bound workloads, programs with random control flow, and very short processes fall outside the validated scope. Fifteen failure modes are documented in the companion negative-results section.

1.5.6 Reproducibility

Full source code, raw experimental data from all 90 runs, and a step-by-step reproduction protocol are publicly available. A Docker container provides bit-for-bit exact reproduction against a pinned environment. The authoritative reproduction command is:

```
git clone https://github.com/rajames440/StarForth.git
cd StarForth
make fastest
./build/amd64/fastest/starforth --doe
```

Deviations from the expected 0.00% algorithmic CV should be reported as bugs.

1.5.7 Summary

- An adaptive virtual machine can achieve 0.00% algorithmic variance across 90 experimental runs.
- Adaptation and determinism are not mutually exclusive; statistical inference drives optimization without introducing non-determinism.
- Performance improves measurably (25.4%) as the system converges to steady state.
- All data, code, and reproduction instructions are publicly available for independent verification.

1.6 Peer Review Notes

Chapter 2

Mathematical Companion

2.1 Q48.16 Fixed-Point Arithmetic

2.2 Feedback Loop Equations

2.3 Figures

Figure 2.1: Phase-space trajectory (snake plot).

Chapter 3

Formal Claims, Anti-Claims, and Null Hypothesis

3.1 Formal Claim Table

3.2 Formal Claim Table

This section provides a structured claim-evidence-reproducibility mapping for peer review. Each primary claim carries a unique identifier, a falsifiable threshold, and a one-command reproducibility protocol. Section-level cross-references connect each claim to its supporting documentation.

3.2.1 Primary Claims

Table 3.1: Primary claims with evidence locations and falsification thresholds.

ID	Statement	Reproducible?	Falsification thre
C1	Algorithmic CV = 0.00% in cache decisions	Yes	CV > 0.1%
C2	25.4% performance convergence (C_FULL)	Yes	No improvement, p
C3	Variance separation: algorithm (0%) vs. environment (70%)	Yes	Algorithm CV > 0
C4	Same workload → same cache configuration, 90 runs	Yes	Any run differs
C5	$p < 10^{-30}$ for determinism (F-test)	Yes	$p > 0.05$

3.2.2 Claim C1: Algorithmic Determinism

Full statement. The adaptive runtime exhibits 0.00% coefficient of variation in algorithmic decisions (cache hit rates, dictionary lookup paths) across 30 identical runs, demonstrating determinism in the adaptive mechanism despite environmental stochasticity.

Table 3.2: Evidence for Claim C1.

Type	Location	Key value
Raw data	`03_EXPERIMENTAL_DATA/full_90_run_comprehensive/`	Cache hit rates
Summary stats	`experiment_summary.txt`	$\mu = 17.39\%$, $\sigma = 0.00\%$
Statistical test	F-test, variance homogeneity	$F(29, 29) \to \infty$, $p < 10^{-30}$
Replication	Git commit SHA checksums	`EXPERIMENTAL_DATA_CHECKSU`

Evidence chain.

Controlled confounds. CPU governor set to `performance`; Turbo Boost disabled; ASLR disabled; process pinned to core 0.

Reproduction.

```
make fastest && ./build/amd64/fastest/starforth --doe --config=
    C_FULL
# Expected: Cache hit rate = 17.39 +/- 0.00%
```

Falsification criteria.

- Independent replication yields CV $> 0.1\%$.
- Any single run shows cache decisions differing from others.
- Environmental perturbation (thermal stress) changes the cache configuration.

Defense. "Measurement noise" objection: 70% CV in runtime demonstrates the timer works; if measurement were inadequate, runtime would also show 0% CV. "Lucky data" objection: probability of coincidence across 90 runs is $< 10^{-30}$ under the null model. "Trivial workload" objection: Fibonacci(20) generates $\approx 2.1 \times 10^6$ word executions with non-trivial recursion.

3.2.3 Claim C2: Adaptive Convergence

Full statement. Configuration C_FULL (all adaptive mechanisms active) demonstrates statistically significant performance convergence of $25.4 \pm 1.2\%$ between early runs (1–15) and late runs (16–30), while non-adaptive configurations show no improvement.

Table 3.3: Convergence results by configuration.

Config	Early runs	Late runs	Improvement	p-value
C_NONE (baseline)	10.76 ms	11.45 ms	−6.4% (degradation)	$p > 0.10$
C_CACHE (moderate)	7.84 ms	7.80 ms	+0.5% (stable)	$p > 0.80$
C_FULL (adaptive)	10.20 ms	7.61 ms	+25.4%	$p < 0.001$

Evidence chain. Statistical test: two-sample t-test ($t = 4.23$, df $= 28$, $p = 0.00012$). Effect size: Cohen's $d \approx 5.08$ (large).

Control logic. C_NONE degrades (no optimization); C_CACHE stabilizes (warmup, not adaptation); only C_FULL converges. This three-way comparison isolates the adaptation-specific effect.

Falsification criteria.

- C_FULL shows no improvement ($p > 0.05$).
- C_NONE shows equal or better convergence than C_FULL.
- Improvement is within margin of error (Cohen's $d < 0.2$).

3.2.4 Claim C3: Variance Separation

Full statement. Variance decomposition reveals algorithmic variance (cache decisions) of 0.00% CV while environmental variance (wall-clock runtime) exhibits 60–70% CV. The two components are statistically independent.

Table 3.4: Variance decomposition.

Component	CV	Source	Measurement
Algorithmic (cache decisions)	0.00%	Deterministic decisions	Integer counters
Environmental (runtime)	60–70%	OS scheduler, thermal	Wall-clock timer
Independence	Pearson $r = 0.03$, $p = 0.87$ (no correlation)		

Falsification criteria.

- Cache decisions correlate with runtime variance ($r > 0.3$).
- Environmental perturbation affects cache CV.
- Algorithmic CV increases under OS load.

3.2.5 Claim C4: Reproducibility

Full statement. Identical workload execution produces bit-for-bit identical adaptive runtime state (cache configuration, frequency rankings, window metrics) with 100% reproducibility across all 90 experimental runs.

Sources of determinism.

- No random number generators anywhere in the production execution path.
- $\mathbb{Q}_{48.16}$ fixed-point arithmetic throughout (no IEEE-754 non-determinism).
- `CLOCK_MONOTONIC_RAW` time source (unaffected by NTP).
- Rolling window seeded from execution history; same history yields same metrics.

Reproduction.

```
./starforth --doe --config=C_FULL > run1.csv
./starforth --doe --config=C_FULL > run2.csv
diff run1.csv run2.csv
# Expected: no differences
```

Falsification criteria.

- Any two runs with identical workload produce different cache configurations.
- Floating-point non-determinism is observed on different CPUs.
- Replication on a different machine yields a different steady state.

3.2.6 Claim C5: Statistical Significance

Full statement. The observed 0.00% CV in algorithmic variance is statistically significant at $p < 10^{-30}$, rejecting the null hypothesis (variance due to chance) with overwhelming confidence.

Power analysis ($n = 30$, $\delta = 0.1$, $\sigma = 0.05$, $\alpha = 0.05$) yields power > 0.99: if 0.1% variance existed, the experiment would have detected it.

Table 3.5: Statistical tests for Claim C5.

Test	**Statistic**	p**-value**	**Interpretation**
F-test (variance homogeneity)	$F(29, 29) \approx \infty$	$p < 10^{-30}$	Reject H_0
Levene's test (robustness)	$W = 0.00$	$p < 10^{-20}$	Reject H_0
Bayesian posterior	$P(H_1 \mid \text{data})$	$\approx 1 - 10^{-30}$	H_1 virtually certain

Falsification criteria.

- Independent analysis yields $p > 0.05$.
- Power analysis shows $N = 30$ insufficient.
- Bayesian posterior $P(H_1 \mid \text{data}) < 0.95$.

3.2.7 Supporting Claims

Table 3.6: Secondary supporting claims.

ID	**Statement**	**Reproducible?**
S1	FORTH-79 compliance (780^+ tests pass)	Yes
S2	Zipf-law execution distribution ($\alpha \approx 1.1$)	Yes
S3	Exponential decay model fit ($R^2 > 0.95$)	Yes
S4	Window inference via Levene's test functional	Yes
S5	Heartbeat coordination operational	Yes

3.2.8 Claim Dependency Structure

Claim C5 establishes statistical validity; C1 is the foundational claim (determinism); C2 and C3 depend on C1 (convergence requires determinism; variance separation presupposes it); C4 validates all claims through end-to-end reproducibility. If C1 falls, all claims fall. If C5 falls, claims become anecdotal.

3.2.9 Patent-Relevant Claims

Table 3.7: Claims with patent relevance. No claim language is drafted here.

ID	**Topic**	**First public disclosure**	**Notes**
P1	Rolling Window of Truth	2025-12-13	Novel; no prior art identified
P2	Deterministic statistical inference	2025-12-13	Application novel; ANOVA standard
P3	Thermodynamic metaphor	N/A	Not patentable (conceptual framework)
P4	Hot-words cache	2025-12-13	Prior art: Ertl (1996); claim: deterministi

3.2.10 Using This Table in Review

When a reviewer challenges a specific claim, identify the claim ID from Table 3.1, cite the evidence location in the row above, and offer the falsification test as the definitive resolution mechanism. The expected response structure is: acknowledge the concern, point to the claim ID, cite the evidence location, offer the reproduction command, and address the specific objection with the defense strategy documented above.

3.3 Anti-Claims

3.4 Out-of-Scope Claims: What This Work Does Not Assert

This section enumerates claims the StarForth project explicitly does *not* make. Its purpose is to bound the scope of peer review: a reviewer who attributes an unstated claim to the work is engaging a strawman. Each entry pairs what the implementation *does* assert with what it *does not* assert. The final summary table (§3.4.11) is the canonical reference for authors responding to review challenges.

3.4.1 Physics and Thermodynamics

Not claimed: a physical theory. The adaptive runtime uses execution frequency as a proxy for thermal energy and exponential decay as a model of heat dissipation. These are conceptual tools borrowed from thermodynamics, not claims that physical laws govern code execution. No actual thermal processes occur in the CPU as a consequence of this model; no quantum effects are invoked; energy conservation in the thermodynamic sense does not apply to execution frequency.

Precise language: *"execution frequency evolves like heat in a cooling system,"* not *"execution frequency is heat."*

Not claimed: frequency is thermal energy. The implementation uses a 64-bit integer counter (`uint64_t execution_heat`) incremented on each word execution. It does not measure joules, calories, or CPU die temperature. The term "heat" is a metaphorical label. Exponential decay *resembles* heat dissipation and steady-state convergence *mirrors* thermodynamic equilibrium in a structural, mathematical sense only.

3.4.2 Optimality and Performance

Not claimed: global optimality. The system demonstrates a 25.4% performance improvement over baseline under the conditions described. This does not imply that no other adaptive runtime could perform better, that optimality is proven mathematically, or that the implementation outperforms all other virtual machines. The contribution is *a working adaptive system*, not the globally optimal one.

Not claimed: superiority to JIT compilers. JIT compilers such as PyPy, LuaJIT, and HotSpot share the conceptual goal of frequency-based specialization. The claim here is not superior raw speed but *deterministic adaptation*: the same adaptive decisions occur on every run, enabling formal verification of the optimization mechanism itself. JITs typically cannot make this guarantee.

Not claimed: universal workload applicability. The system is validated on CPU-bound, deterministic FORTH programs, including recursive algorithms (Fibonacci, Ackermann) across workload shapes with Zipf exponent $\alpha \in [0.8, 1.5]$. It does not claim to improve I/O-bound workloads, non-deterministic programs, adversarial execution patterns, or very short-lived processes (fewer than approximately 1,000 iterations). Explicit failure modes are documented in the companion negative-results section.

3.4.3 Machine Learning and Artificial Intelligence

Not claimed: AI or machine learning. The adaptive mechanism uses statistical inference (ANOVA, Levene's test, exponential regression) and data-driven parameter tuning. No neural

networks, gradient descent, backpropagation, training datasets, deep learning, or reinforcement learning are involved.

Precise language: *"statistically-inferred adaptive tuning,"* not *"AI-driven optimization."*

Not claimed: the system learns. The system *adapts*: parameters converge to a steady state and feedback loops stabilize metrics. This is statistical convergence, not supervised or unsupervised learning. Knowledge does not transfer between workloads; the steady state is workload-specific.

3.4.4 Novelty and Prior Art

Not claimed: first adaptive virtual machine. Execution frequency tracking is standard profiler practice; hot-code caching is the basis of every production JIT; exponential decay underlies LRU eviction. The novelty claim is specific: the *combination* of adaptation with 0% algorithmic variance and formal verification potential, which prior systems do not offer.

Not claimed: a replacement for JIT compilation. The implementation targets a different niche—verifiable adaptive systems for safety-critical contexts—rather than competing directly with LLVM or V8. Compilation remains necessary for applications requiring peak throughput.

3.4.5 Formal Verification

Not claimed: complete formal verification. Convergence theorems are stated and empirically validated (0% coefficient of variation across 90 runs). Full mechanized proofs for all seven feedback loops in Coq or Isabelle are *not* claimed; partial proofs are in progress.

Precise language: *"empirically validated determinism,"* not *"formally proved correct."*

Not claimed: zero bugs. The test suite comprises 936$^+$ tests and validates FORTH-79 compliance. No known correctness bugs exist in the core interpreter as of the experimental baseline commit. This provides high assurance, not mathematical proof of defect-absence.

3.4.6 Causation and Interpretation

Not claimed: proven causation. The association between adaptive mechanisms and the observed 25.4% improvement is strong: the functional relationship fits an exponential decay model with $R^2 > 0.95$, and only the fully adaptive configuration (`C_FULL`) improves. No randomized controlled trial establishes mechanistic causation.

Precise language: *"adaptive mechanisms are associated with 25.4% improvement,"* not *"cause 25.4% improvement."*

Not claimed: a theory of computation. The work provides a VM optimization technique and a predictive framework for adaptive runtime performance. It does not propose a fundamental theory of computation, replace computational complexity theory, or define a new model of computation.

3.4.7 Generalization

Not claimed: cross-language generalization. Principles may generalize to other interpreter architectures (Lua, Python, JavaScript), but this has not been validated. Compiled languages are explicitly out of scope: ahead-of-time optimization already handles hot-code without runtime frequency tracking.

Not claimed: arbitrary scalability. The system is validated on programs up to approximately 2.1 million word executions with dictionary sizes up to approximately 500 entries. Scalability to programs with 100,000$^+$ dictionary entries is not tested; transition-matrix memory overhead would grow quadratically in that regime.

3.4.8 Deployment Status

Not claimed: production readiness. StarForth is a research prototype suitable for experimental validation. It demonstrates feasibility of deterministic adaptation and serves as a platform for further study. It is not a production-grade implementation with enterprise support or hardening against all security threats.

Not claimed: an operating system. The StarForth → StarKernel → StarshipOS sequence is a roadmap, not a set of delivered products. StarshipOS does not yet exist as a functional system; the vision is aspirational.

3.4.9 Statistical Claims

Not claimed: zero variance in all metrics. The 0.00% coefficient of variation applies to *algorithmic decisions*: cache hit rates and dictionary lookup paths. Wall-clock runtime exhibits 60–70% CV due to OS scheduler noise, thermal variation, and cache-line effects. These two components are statistically independent (Pearson $r = 0.03$, $p = 0.87$).

Precise language: *"algorithmic variance: 0.00% CV,"* not *"total system variance: 0.00% CV."*

Not claimed: 100% confidence. Results are reported at 95% confidence intervals. The Bayesian posterior $P(H_1 \mid \text{data}) \approx 1 - 10^{-30}$ is effectively certain but not unity. Science deals in probabilities, not absolute certainties; replication failure, while astronomically unlikely, is not logically impossible.

3.4.10 Scope Exclusions

Not claimed: solutions to undecidable problems. Adaptive convergence to steady state is asserted only for *terminating* programs with analyzable workloads. No claim is made about decidability of convergence for arbitrary programs.

Not claimed: quantum or blockchain components. The implementation uses classical algorithms on classical hardware. No quantum superposition, entanglement, distributed ledger, cryptocurrency, smart contracts, or neuromorphic computing is involved.

Not claimed: a performance record over any named system. No direct performance shootout against PyPy, LuaJIT, or HotSpot is presented. The contribution is deterministic adaptation, not a speed record.

3.4.11 Summary Table

3.4.12 How to Use This Section in Peer Review

When a reviewer attributes a claim not appearing in the formal claim table, the appropriate response is to cite the relevant paragraph above by section number. If the attributed claim does not appear in this section either, it may represent a genuine novel claim; in that case, authors should locate the supporting evidence in the formal claim table before responding.

Table 3.8: Anti-claims reference table. Left column: what the work asserts. Right column: what it explicitly does not assert.

Category	**Asserted**	**Not Asserted**
Physics	Thermodynamic metaphor	Actual physical theory
Performance	25.4% improvement	Global optimality
Novelty	Deterministic adaptation	First adaptive system
Verification	Empirical validation	Complete formal proof
ML/AI	Statistical inference	Neural networks or learning
Causation	Strong correlation	Proven causation
Generality	Works for FORTH	Works for all languages
Variance	Algorithmic: 0% CV	Total system: 0% CV

The intellectual commitment underlying this section is that explicit scope-bounding prevents both strawman attacks and over-interpretation by readers. Transparency about limitations strengthens, rather than weakens, the credibility of the work.

3.5 Null Hypothesis and Falsification Criteria

3.6 Null Hypothesis and Falsification Framework

3.6.1 Statement of the Null Hypothesis

H_0 **(Null):** Performance variance in StarForth is attributable solely to environmental stochasticity—OS scheduling noise, cache effects, thermal throttling—and no invariant scaling relationship exists between adaptive mechanisms and steady-state metrics. Formally:

$$H_0 : \sigma_{\text{algorithmic}} = \sigma_{\text{environmental}}$$

The algorithm contributes no determinism beyond measurement noise.

H_1 **(Alternative):** Adaptive mechanisms produce deterministic cache decisions distinct from environmental noise. Formally:

$$H_1 : \sigma_{\text{algorithmic}} < \sigma_{\text{environmental}}$$

3.6.2 Evidence Against the Null Hypothesis

Variance decomposition. Measured across 90 experimental runs:

Table 3.9: Variance decomposition from the 90-run experiment.

Metric	**Algorithm CV**	**Environment CV**
Cache hit rate	0.00%	N/A
Wall-clock runtime	N/A	60–70%
Statistical independence	Pearson $r = 0.03$, $p = 0.87$	

Statistical test. Two-sample F-test for variance homogeneity yields $F(29, 29) \approx \infty$, $p < 10^{-30}$. H_0 is rejected at $\alpha = 0.05$.

3.6.3 What Would Falsify the Claims

Claim 1 (algorithmic determinism).

- Cache hit rates vary by more than 0.1% across identical runs.
- Dictionary lookup decisions differ between runs with identical workloads.
- Rolling window history contains non-deterministic elements.

Empirical test:

```
for i in {1..100}; do
    ./starforth --doe --config=C_FULL > run_${i}.csv
done
# Falsified if CV(cache_hits) > 0.1%
```

Claim 2 (adaptive convergence).

- C_FULL shows no improvement over 30 runs ($p > 0.05$).
- C_NONE shows equal or better convergence than C_FULL.
- Late-run performance is statistically indistinguishable from early-run performance.

Claim 3 (stability under environmental noise).

- Algorithm variance scales proportionally with environmental variance.
- OS scheduling noise propagates into cache decisions.
- External perturbations (thermal throttling) change the cache configuration.

Falsification under thermal stress:

```
stress-ng --cpu 8 --timeout 60s &
./starforth --doe --config=C_FULL > stressed.csv
# Falsified if cache CV under stress > 0.5%
```

Claim 4 (reproducibility).

- An independent researcher cannot reproduce CV = 0.00%.
- Different hardware produces significantly different convergence rates.
- Replication across systems yields conflicting results.

3.6.4 Null Model Predictions

Under H_0, the expected observations are:

- Cache hit rates vary randomly at 10–20% CV (typical scheduling noise). *Observed: 0.00%.* Null model fails.
- All configurations show similar adaptation curves. *Observed: only C_FULL converges.* Null model fails.
- Runtime CV ≈ cache CV. *Observed: 70% vs. 0%.* Null model fails.
- No fixed-point attractor in phase space. *Observed: stable convergence.* Null model fails.

The null model fails on all four predictions.

3.6.5 Bayesian Interpretation

With an agnostic prior ($P(H_0) = P(H_1) = 0.50$), the likelihood ratio from 90 runs is approximately $P(\text{data} \mid H_1)/P(\text{data} \mid H_0) \approx 10^{30}$. The posterior is:

$$P(H_1 \mid \text{data}) \approx 1 - 10^{-30}$$

The null hypothesis is astronomically implausible given the observed data.

3.6.6 Control Experiments

Positive control (should show variance). Introducing a timer-based random element (`TIMER @ 12345 XOR`) breaks determinism and produces CV $> 0\%$. This proves the measurement infrastructure can detect variance when it exists.

Negative control (should show determinism). A minimal FORTH program (`: SIMPLE 1 2 + . ;`) with no adaptation produces CV $= 0.00\%$—the trivial case. This establishes the measurement noise floor.

3.6.7 Statistical Power

With $n = 30$ runs, $\delta = 0.1$ (minimum detectable CV difference), $\sigma = 0.05$, and $\alpha = 0.05$, the computed power exceeds 0.99. If any non-determinism as small as 0.1% CV existed, the experiment would have detected it with 99% probability. The experiment is over-powered for the detection task.

3.6.8 Alternative Explanations Considered

Alternative 1: measurement artifact. If measurement resolution were inadequate, runtime would also show 0% CV. Runtime shows 70% CV. This alternative is rejected.

Alternative 2: cherry-picked data. All 90 runs are committed to the repository with SHA256 checksums. No runs were excluded. The experimental protocol was documented before data collection. This alternative is rejected.

Alternative 3: coincidental stability. Stability is observed across three configurations, 90 runs spanning multiple days, and deliberate thermal stress. Probability of coincidence across all conditions is $< 10^{-30}$. This alternative is rejected.

Alternative 4: trivial workload. Fibonacci(20) generates $\approx 2.1 \times 10^6$ word executions with recursive control flow and power-law execution distribution (Zipf $\alpha \approx 1.1$). This is a partially valid concern: generalization to all workloads requires further study, and I/O-bound or random workloads are explicitly out of scope (see §3.8).

3.6.9 Burden of Proof

To reject the claims, a skeptic must produce at least one of:

1. An independent replication with CV $> 0.1\%$ under controlled conditions.
2. A demonstrated measurement artifact producing false 0% CV.
3. Evidence that the statistical analysis is fundamentally flawed.
4. An alternative explanation consistent with all evidence simultaneously.

Absent one of these, the null hypothesis is rejected and the claims stand. Independent replication protocols are provided in §??.

3.7 Negative Results

3.8 Negative Results: Documented Failure Modes

Systems that never fail are artifacts, not scientific results. This section documents 15 failure modes identified through deliberate adversarial testing. These are not accidental discoveries; experiments were designed to break the system. The failure modes bound the system's applicability and constitute essential context for interpreting the positive claims.

A concise summary table appears at the end of this section (§3.8.8).

3.8.1 Workload-Dependent Failures

Failure mode 1: random execution paths. A workload in which control flow depends on a nanosecond timer (`TIMER @ 2 MOD`) produces non-reproducible execution sequences. Cache hit rate CV rises to approximately 15%; execution heat distributions are non-reproducible; the system oscillates without converging. Root cause: the rolling window captures different execution sequences on each run, giving the adaptive mechanisms a moving target. No mitigation exists; this is an intentional scope boundary. Deterministic adaptation requires deterministic workloads.

Failure mode 2: I/O-bound workloads. A workload dominated by file I/O (1,000 iterations of `OPEN-FILE / READ-FILE / CLOSE-FILE`) achieves at most 2% improvement, within margin of error. The adaptive mechanisms add approximately 5% overhead, producing net degradation. Root cause: the bottleneck is disk latency, not dictionary lookup; I/O timing variance swamps algorithmic determinism. Mitigation (not yet implemented): detect I/O-bound workloads and disable adaptive loops.

Failure mode 3: short programs. Programs with fewer than approximately 1,000 word executions provide insufficient data for statistical inference. Levene's test requires minimum sample sizes; exponential regression on three data points is meaningless. Overhead exceeds benefit. Mitigation: disable adaptive loops when `execution_count < THRESHOLD` (suggested: 10,000).

Failure mode 4: adversarial workloads. A workload that executes each word exactly once in rotation produces a flat frequency distribution—the Zipf-law assumption is violated. The hot-words cache is useless (all words equally "hot"); performance improvement is 0%. Mitigation: detect flat distributions via entropy measurement and disable the hot-words cache.

3.8.2 Parameter-Dependent Failures

Failure mode 5: window size too small. Setting `ROLLING_WINDOW_SIZE = 10` (default: 4,096) produces unstable variance inflection detection ($R^2 < 0.5$) and convergence oscillation. Statistical noise dominates the signal. Mitigation: enforce a minimum window size of at least 1,024 entries.

Failure mode 6: decay slope too steep. Setting $\lambda = 10.0$ (default: ≈ 0.001) causes cache thrashing: words are promoted and immediately demoted, the system never reaches steady state, and performance degrades below baseline. Root cause: steep decay erases long-term frequency

information, making the cache reactive to noise rather than signal. Mitigation: bound decay slope to $\lambda \in [10^{-4}, 10^{-2}]$.

Failure mode 7: cache size mismatch. Setting `HOTWORDS_CACHE_SIZE = 1` (default: 16) reduces cache hit rate to approximately 5% and performance improvement to approximately 2%. Root cause: the test workload contains 10–15 hot words; a cache of 1 entry captures only the single hottest. Mitigation: auto-tune cache size based on workload entropy.

3.8.3 Environmental Failures

Failure mode 8: thermal throttling. Sustained CPU load sufficient to trigger thermal throttling increases runtime CV to 90^{+}% and extends the convergence window from ≈30 to ≈40 runs. Cache decisions remain at 0% CV: timing does not affect algorithmic choices. Root cause: wall-clock-based decay becomes inconsistent when the CPU reduces its clock frequency. Mitigation: use CPU cycle counters instead of wall-clock timers for decay application.

Failure mode 9: aggressive OS preemption. Running with 32 concurrent CPU-saturating background processes pushes runtime CV above 150% and may prevent convergence if the process is preempted too frequently for decay to apply correctly. Cache decisions remain at 0% CV. Mitigation: pin the process to an isolated core and set real-time priority.

3.8.4 Architectural Failures

Failure mode 10: cross-architecture performance difference. Cache decisions are bit-identical across x86_64 (Intel Xeon) and AArch64 (ARM Cortex) because the algorithm is purely arithmetic. Convergence *rate* differs—Intel achieves approximately 25% improvement, ARM approximately 18%—because dictionary lookup has a different relative cost on each architecture. This is expected behavior, not a failure of determinism. No mitigation is needed; cross-architecture performance comparisons require hardware-normalized baselines.

Failure mode 11: floating-point non-determinism (hypothetical). IEEE-754 arithmetic is not used anywhere in the adaptive runtime; $\mathbb{Q}_{48.16}$ fixed-point arithmetic provides bit-identical results across architectures and compilers. This entry documents why floating-point was avoided: FMA instructions, compiler reordering, and denormal handling all introduce non-determinism that would violate the 0% CV claim.

3.8.5 Implementation Bugs Resolved

Bug 1: heartbeat thread race condition (fixed). An unsynchronized shared-state access between the heartbeat thread and the main interpreter thread caused segmentation faults in fewer than 1% of runs. Fix: added a mutex around `vm_tick()` (commit `8133787`). The experimental data was collected after this fix.

Bug 2: integer overflow in execution heat (fixed). 32-bit frequency counters overflowed on long-running programs, causing execution frequency to reset to zero unexpectedly. Fix: counters promoted to `uint64_t` (commit `c0ec82d`).

3.8.6 Statistical Failures

Failure mode 12: insufficient sample size. With $N = 5$ trials instead of the $N = 30$ used in the experiments, the Central Limit Theorem does not apply; confidence intervals are unreliable; t-test normality assumptions are violated; statistical power falls below 50%. Mitigation: require $n \geq 30$ for all experiments.

Failure mode 13: multiple testing without correction. Testing 100 hypotheses at $\alpha = 0.05$ without Bonferroni correction yields approximately 5 spurious positives by chance. The formal claim table tests 5 primary claims; corrected $\alpha = 0.05/5 = 0.01$. All reported p-values remain significant under this correction.

3.8.7 Generalization Failures

Failure mode 14: compiled languages. The adaptive runtime is an interpreter optimization. Ahead-of-time compilers already perform static frequency analysis and hot-code optimization without runtime overhead; the rolling window and heartbeat infrastructure would add cost with no benefit. This is a scope boundary, not a bug.

Failure mode 15: very large programs. Programs with $100{,}000^{+}$ dictionary entries would require sorting a 100K-entry array on every heartbeat tick and maintaining a 100K $\times$ 100K transition matrix—both computationally prohibitive. Scalability at this scale is unvalidated; it is marked as future work.

3.8.8 Summary of Failure Modes

Table 3.10: Failure mode summary. All failures are predictable and bounded.

Mode	Root cause	Mitigation
Random execution paths	Non-deterministic workload	Detect and disable
I/O-bound program	Wrong bottleneck	Profile first
Short program	Insufficient data	Minimum execution threshold
Adversarial (flat) workload	No frequency skew	Detect and disable cache
Window size too small	Insufficient context	Enforce minimum: 1,024
Decay too steep	Premature forgetting	Bound $\lambda \in [10^{-4}, 10^{-2}]$
Cache too small	Mismatched capacity	Auto-tune to workload entropy
Thermal throttling	Inconsistent clock	Use cycle counters
OS preemption	Context switches	Isolate core, RT priority
Cross-architecture	Hardware differences	Expected; document
Floating-point (hypothetical)	IEEE-754 rounding	Fixed-point throughout
Small sample ($N < 30$)	CLT does not apply	Require $n \geq 30$
Multiple testing	Inflated α	Bonferroni correction
Compiled languages	Wrong paradigm	Out of scope
Massive programs	Untested regime	Future work

3.8.9 Lessons

Five lessons emerge from the failure modes:

- Determinism requires deterministic inputs: random workloads break all adaptive guarantees.
- Statistical inference requires sufficient data: short programs and small sample sizes produce meaningless results.
- Optimization requires locality: workloads with flat frequency distributions have nothing to cache.

- Architecture matters: floating-point and thermal effects are real engineering constraints, not theoretical concerns.

- Scope boundaries make positive results credible: a system claiming unlimited applicability invites justified skepticism.

The claims in §3.2 are scoped to CPU-bound, deterministic, long-running FORTH programs. Outside that scope, the system fails predictably and in ways documented above.

3.9 Academic Wording Guidelines

3.10 Academic Wording Guidelines

Careful language discipline is essential when a system uses thermodynamic quantities as conceptual tools rather than as physical claims. This section codifies the approved vocabulary for all publications and documentation derived from the StarForth adaptive runtime research, and provides sentence patterns and review-response templates for common challenge scenarios.

3.10.1 The Core Rule: Similarity, Not Equivalence

Mathematical similarities between StarForth's empirical results and equations drawn from physics are described with language that asserts structural resemblance, not physical identity. The distinction is not cosmetic: a claim of equivalence invites physicists to evaluate the work as a physics paper (which it is not), while a claim of structural similarity accurately characterizes the mathematical relationship and places the analogy in the domain where it belongs—applied statistics and systems modeling.

3.10.2 Approved Phrases

Describing mathematical relationships. The preferred formulation evaluates multiple candidate models before selecting one:

> "Multiple candidate models were evaluated—linear, polynomial, exponential, and power-law—and the functional form $f(t) = f_0 e^{-\lambda t}$ provided the best empirical fit with $R^2 = 0.96$."

This construction defends against cherry-picking accusations by establishing that alternatives were tested. Approved phrases for describing the selected fit include:

- *fits the functional form*

- *shares structural similarity with*

- *is mathematically isomorphic to*

- *exhibits the same mathematical structure as*

- *the empirical data conforms to*

- *matches the pattern of*

Describing observations. Empirical statements should lead with the measurement, not the model:

- *The data reveals...*
- *Measurements indicate...*
- *The experimental results demonstrate...*

Noting physics parallels. When drawing a connection to a physics equation or law, use:

- *shares dimensional similarity with*
- *the functional form also appears in [physics context]*
- *this mathematical structure is also found in*
- *bears mathematical resemblance to*

3.10.3 Forbidden Phrases

The following constructions imply equivalence or causation and must not appear outside explicitly labelled interpretation sections.

Direct equivalence claims.

- "is equivalent to" — use *fits the functional form*
- "is the same as" — use *shares structural similarity with*
- "proves that [physics concept] applies" — use *the data is consistent with*

Unqualified analogies.

- "is analogous to" (without the qualifier "structurally")
- "corresponds to" (without "structurally" or similar)
- "behaves as" (without qualification)
- "mirrors" (implies direct correspondence)
- "exactly as predicted by" (implies causal application of theory)

Causation implications.

- "because of [physics principle]"
- "due to [physics concept]"
- "governed by [physics law]"
- "obeys [physics equation]"

3.10.4 Before-and-After Examples

Example 1: performance scaling. Incorrect:

> "Performance scales exactly as predicted by special relativity's time dilation."

Correct:

> "Multiple candidate performance models were evaluated. The empirical data fits the functional form $\tau = \tau_0/\sqrt{1-\beta^2}$, which shares structural similarity with the Lorentz transformation from special relativity."

The correct version (1) establishes that alternatives were evaluated, (2) uses "fits the functional form" as an empirical observation, and (3) uses "shares structural similarity" as a mathematical fact, not a causal claim.

Example 2: window capacity. Incorrect:

> "The system obeys a cosmological constant law $\Lambda(\text{DoF}) = 4096/(\text{DoF} + 1)$."

Correct:

> "Empirical measurements reveal that window capacity follows the relationship $\Lambda(\text{DoF}) = 4096/(\text{DoF} + 1)$ with 0.00% coefficient of variation. The symbol Λ is borrowed from cosmology for mathematical convenience."

The correct version uses "empirical measurements reveal" and "follows the relationship" (descriptive, not normative), and explicitly labels Λ as notation borrowed for convenience.

Example 3: geodesic convergence. Incorrect:

> "Workloads converge along geodesics analogous to general relativity."

Correct:

> "All workload trajectories converge to the same attractor basin regardless of starting conditions. This pattern shares structural similarity with geodesic motion in curved spacetime."

The correct version states the empirical behavior first, then draws the mathematical comparison—without implying the physics explains the computation.

3.10.5 Defensive Language Patterns

Three reusable templates cover the most common situations.

Pattern 1: model evaluation statement.

> "[N] candidate models were evaluated, including [list]. The functional form [equation] provided the best fit with $R^2 =$ [value]."

Pattern 2: empirical observation plus mathematical note.

> "[Empirical observation]. This [relationship / pattern / structure] also appears in [context] as [reference]."

Pattern 3: metaphor disclosure.

> "A thermodynamic metaphor is employed, where execution frequency decreases over time analogously to heat dissipation. The implementation uses exponential decay $f(t) = f_0 e^{-\lambda t}$ applied at each heartbeat tick."

This pattern explicitly labels the metaphor and immediately pairs it with a literal description of the implementation.

3.10.6 Section-Specific Guidelines

Abstracts and titles Use mathematical or empirical language only. *"Frequency-Based Adaptive Runtime"* is acceptable; *"Physics-Driven Adaptive Runtime"* is not, in a strict academic context.

Introductions State empirical observations before drawing any mathematical comparison. Never assert that physics explains computation.

Methods and implementation sections Use literal descriptions. *"Frequency counter incremented on execution"* is preferred over *"temperature increases when word heats up."* The metaphorical vocabulary is reserved for conceptual exposition, not implementation description.

Results sections Use statistical language. Report *"CV = 0.00% ($p < 10^{-30}$)"* rather than *"perfect thermodynamic equilibrium achieved."*

Discussion and interpretation sections Exploratory language is permitted with appropriate qualification: *"One interpretation is that... however, causation is not established."* The phrase *"this proves that computation follows physics laws"* is never acceptable.

3.10.7 Reviewer Challenge Responses

"You are claiming physics." The appropriate response cites the section where the thermodynamic framework is explicitly introduced as a conceptual metaphor, then points to the mathematical claim: the functional form $f(t) = f_0 e^{-\lambda t}$ fits performance data with $R^2 = [\text{value}]$. The fact that this functional form also appears in physics is noted as a mathematical similarity, not a physical claim.

"You cherry-picked equations." Cite the model evaluation section documenting that linear, polynomial, exponential, and power-law candidates were all evaluated, and that the reported form was selected on empirical fit quality (R^2, AIC) rather than theoretical preference.

"This is pseudoscience." All claims are empirically verifiable. The physics references describe mathematical similarities, documented explicitly as metaphorical framing in the ontology section. Point the reviewer to the formal claim table and the reproduction protocol.

3.10.8 Pre-Publication Checklist

Before submitting any paper, presentation, or externally visible documentation, verify the following substitutions have been applied throughout:

- Replace *"is equivalent to"* with *"fits the functional form"*
- Replace *"is analogous to"* with *"shares structural similarity with"*
- Add the qualifier *"structurally"* to any unqualified *"corresponds to"*

- Replace *"mirrors"* with *"matches the pattern of"*
- Replace *"exactly as"* with *"according to the functional form"*
- Replace *"obeys"* with *"follows the relationship"*
- Replace *"governed by"* with *"characterized by"*
- Verify that all metaphors are explicitly labelled in running text
- Confirm that all empirical claims carry data references
- Confirm that all physics parallels use similarity language

3.10.9 The Golden Rule

Describe what was *observed*; note where the observations *mathematically resemble* known equations; do not claim the physics *causes* the behavior.

The defensible position is: "Measurements in this system fit these mathematical forms. These forms also appear in physics. This similarity may provide useful modeling frameworks." The indefensible position is: "Our system implements physics" or "physics laws govern our system."

Chapter 4

Reproducibility and Independent Replication

4.1 One-Command Reproduction

4.2 Replication Invitation

4.3 Scientific Defense Checklist

4.4 Sensitivity Analysis Design

Chapter 5

Roadmap

5.1 Timeline Summary

Table 5.1: StarshipOS development roadmap

2025 (done)	2026	2027	2028
StarForth VM	StarKernel	StarshipOS	FPGA Hardware
FORTH-79 VM done	Bare metal	Self-hosting	Custom silicon
	M0–M6 done	Storage, net	Feasibility study
	M7 in progress	Multitask, self-compile	

5.2 LithosAnanke Roadmap

- `v1.0.x` — serial-only production (current)
- `v1.5.0` — framebuffer VT100 terminal milestone
- `v2.0.0` — StarForth SDK release

5.3 StarshipOS

5.4 FPGA / Custom Silicon